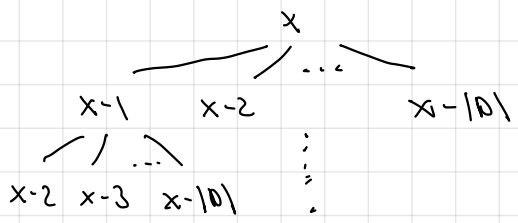
lecture 15
greedy algorithm

monday coin change problem

In terms of x and |D|, what is the worst case not memoized solution

worst case: denominations are $D = \{1, 2, 3, \ldots, |D|\}$

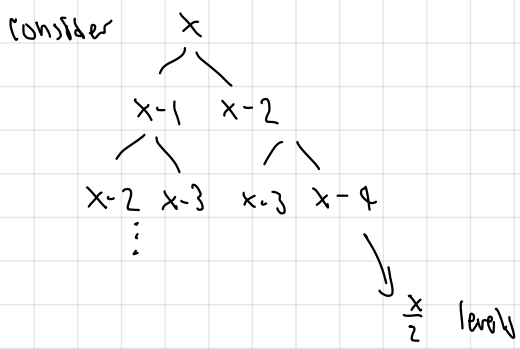


---

lower bound: $\Omega(\sqrt{2}^x \cdot |D|)$

↑ lower bound of nodes ↑ time per call

lower bound:

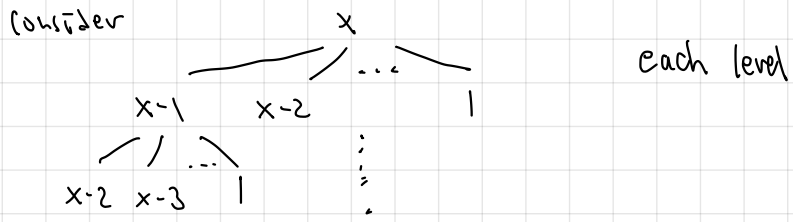


lower bound of 2 nodes per level

$\Omega(2^{\frac{x}{2}}) = \Omega(\sqrt{2}^x)$

---

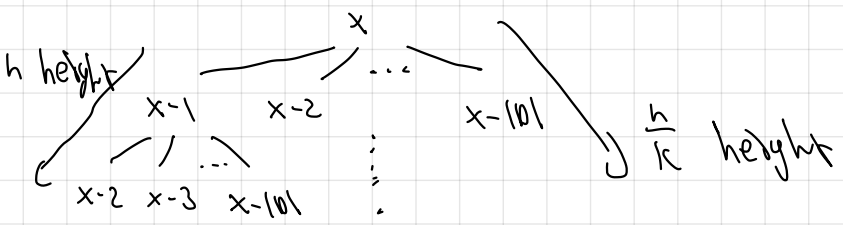
exact count:



recurrence: $T_h = 1 + T_{h-1} + T_{h-2} + \ldots + T_0$

$T_h = 2^{h-1}$

now go until $x - k$. $k = |D|$

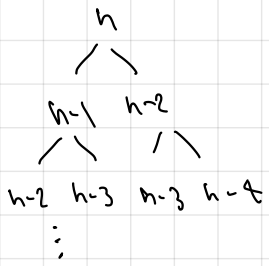


$\Theta\left(\left(\frac{1}{r}\right)^h\right)$

↑ root of $1 - 2z + z^{k+1}$

r approaching $\frac{1}{2}$ as $k \to \infty$

$k = 2$



$= \Theta(1.6^h)$

This is the worst case AVL tree

$T_h = 1 + T_{h-1} + T_{h-2}$

lower bound

$\geq c_h 2 T_{h-2}$

$\geq 2 \cdot 2^{\frac{h}{2}}$

$= \sqrt{2}^h$

recall our lower bound $\sqrt{2}^h \approx 1.4^h$

Combinatorial optimization
- sequence of choices
- each choice has a cost
- minimize global cost

Ideas:
- greedy algorithm
  - local optimal way
- dynamic programming
  - use recursion to find the cost of every option
  - pick the best option
  - use memoization to shrink recursion tree

Reconsider the assumption that greedy algorithm don't work

---

meeting scheduling
- need to go to as many meetings as possible
- can't show up late or leave early

Given a calendar of events
Return optimal set of meetings so they don't overlap and
select as many meetings as possible

Strategy 1 — does not work
- pick the shortest meeting first



The greedy algorithm first picks M_2, but this does not allow us to schedule M_1 nor M_3, which are in the optimal solution

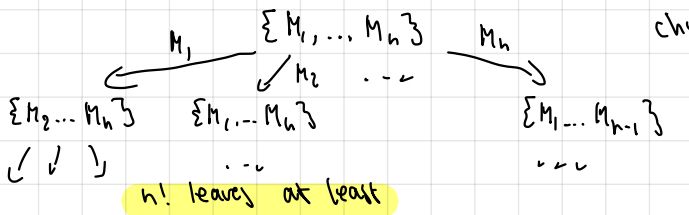
Rules out other option

Now, try dynamic programming

Strategy 2: we make a choice and see it affects the other cases
recursion to evaluate options

DP 1: which meeting should I pick next? $n!$

- Loop through all remaining meetings and try to take each.
- input is remaining meetings



choose next meeting to take

not permutations
not $n!$.
permutations use all elements and order matters

$n!$ leaves at least

- find the best schedule among remaining meetings
- this calculates every subset.
- memoization does not help because there are no repeated calls
- each call is unique

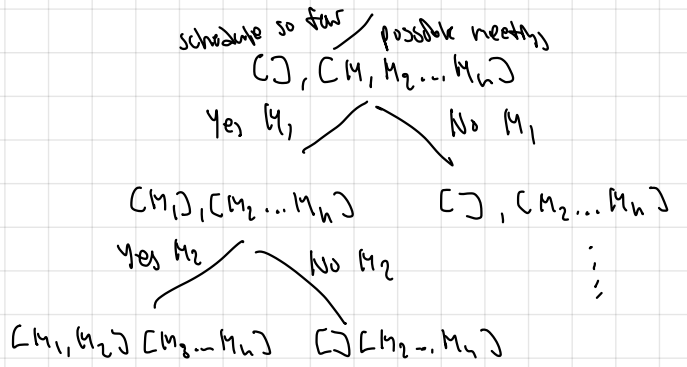
$\geq 2^n$ subsets — unique inputs — $n!$ leaves, since there will be overlapped inputs
- order does not matter
- we don't use every element — also take care of removing overlaps

with memoization, at least 2^n calls

DP 2: Sort meetings by finishing time. Should I take the meeting or not?
Consider 1 meeting as a time

After sorting: $M_1, M_2, \ldots, M_n$

- at each meeting, make a choice yes or no
- start with empty list and keep track of remaining meetings.



2^n subsets and calls

What actually matters is the remaining meetings and those repeat
- the partial schedule does not matter when there is no overlap
- memoize remaining meetings
- for overlap, also need to prevent overlap between so far and possible meeting

$O(n)$ unique inputs
- rethink problem as binary decisions to force more duplicates in the decision tree

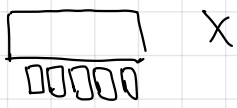
sorting by finishing time will take care of overlap

For DP, the number of unique inputs is what is important

Now, consider greedy algorithms with the following greedy strategies

1. earliest-starting meeting first

Counter: prevents optimal strategy



X

2. earliest-finishing meeting first



$\{M_1, M_3\}$

works

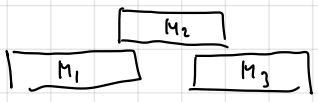
3. latest-finishing meeting first

Counter: prevents optimal strategy



b

4. latest-starting meeting first



$\{M_3, M_1\}$

works

leaves as much free time as possible for remainder
leaves as many open meetings as possible

Iterative Exchange Argument

Consider an optimal schedule

$[S_1, S_2 \dots S_n]$ ordered by time

Let g_1 be the earliest ending of all meetings
↑ sample size bigger than $[S_1 \dots S_n]$

Replace S_1 with g_1

$[g_1, S_2, \dots S_n]$
- same number of meetings as optimal
- no conflicts
  - S_1 must have ended before $[S_2 \dots S_n]$
  - g_1 is the earliest ending meeting, so it must have ended at least before $[S_2 \dots S_n]$

- this is just as good as the optimal with a greedy element

Repeat:

Let g_2 be the earliest ending of all meetings that does not conflict with g_1

Let g_i = earliest ending that does not conflict with $g_1 \dots g_{i-1}$
- what the greedy algorithm finds

One by one, every element in the optimal solution is swapped with the corresponding element in the greedy schedule

ends with greedy solution

size of greedy solution = size of optimal solution



optimal : $[M_1, M_3]$
greedy $[M_1, M_2]$



```
def Schedule(mtgs):
  if no mtgs:
    return []

  earliestend = mtgs[0]
  for mtg in mtgs:
    if mtg ends earlier than earliest end:
      earliestend ∈ mtg

  remaining ∈ {}
  for mtg in mtgs:
    if mtg does not conflict with earliestend:
      remaining ∈ remaining ∪ {mtg}

  return {earliestend} ∪ schedule(remaining)
```

	calls	work	
$O(n)$ calls	n	n	
$O(n)$ time per call	$n-1$	$n-1$	$O(n^2)$
	⋮	⋮	
Total: $O(n^2)$	1	1	

Optimization

- sort by finishing time first $O(n \log(n))$
- next just walk down the array until the next one does not conflict with the previously chosen element/meeting $O(n)$
- repeat until no more meetings

$O(n \log(n))$